

Annotated hexdump: one complete request and response

These bytes were captured live from `./bcurl -v localhost:9000/hello.txt` against `./bserve ./www 9000`.

`www/hello.txt` contains `Hello, binary world!\n`, which is 21 octets. On the wire, the client sends 62 octets and the server sends 121 octets in two frames.

Raw capture (> = client to server, < = server to client)

```
> 00000000 00 00 36 01 01 00 00 01 01 00 03 47 45 54 02 00 |..6.....GET..|
> 00000010 0a 2f 68 65 6c 6c 6f 2e 74 78 74 04 00 0e 6c 6f |./hello.txt...lo|
> 00000020 63 61 6c 68 6f 73 74 3a 39 30 30 30 05 00 09 62 |calhost:9000...b|
> 00000030 63 75 72 6c 2f 31 2e 30 06 00 03 2a 2f 2a      |curl/1.0...*/|

< 00000000 00 00 54 01 00 00 00 01 03 00 03 32 30 30 07 00 |..T.....200..|
< 00000010 19 74 65 78 74 2f 70 6c 61 69 6e 3b 20 63 68 61 |.text/plain; cha|
< 00000020 72 73 65 74 3d 75 74 66 2d 38 08 00 02 32 31 09 |rset=utf-8...21.|
< 00000030 00 0a 62 73 65 72 76 65 2f 31 2e 30 0a 00 1d 54 |..bserve/1.0...T|
< 00000040 68 75 2c 20 30 38 20 4f 63 74 20 32 30 32 36 20 |hu, 08 Oct 2026 |
< 00000050 31 36 3a 35 30 3a 34 35 20 47 4d 54             |16:50:45 GMT|

< 00000000 00 00 15 02 01 00 00 01 48 65 6c 6c 6f 2c 20 62 |.....Hello, b|
< 00000010 69 6e 61 72 79 20 77 6f 72 6c 64 21 0a         |inary world!..|
```

Frame 1: request HEADERS (client to server, 8 + 54 octets)

Offset	Bytes	Field	Meaning
0x00	00 00 36	Length	0x36 = 54 payload octets follow the header
0x03	01	Type	HEADERS
0x04	01	Flags	END: this one frame is the whole request, with no body
0x05	00	Reserved	always 0
0x06	00 01	Request ID	1, the first request on this connection
0x08	01	Index	1 = :method
0x09	00 03	ValueLen	3
0x0b	47 45 54	Value	GET
0x0e	02	Index	2 = :path
0x0f	00 0a	ValueLen	10
0x11	2f 68 65 6c 6c 6f 2e 74 78 74	Value	/hello.txt
0x1b	04	Index	4 = host
0x1c	00 0e	ValueLen	14
0x1e	6c 6f 63 61 6c 68 6f 73 74 3a 39 30 30 30	Value	localhost:9000
0x2c	05	Index	5 = user-agent
0x2d	00 09	ValueLen	9
0x2f	62 63 75 72 6c 2f 31 2e 30	Value	bcurl/1.0
0x38	06	Index	6 = accept
0x39	00 03	ValueLen	3
0x3b	2a 2f 2a	Value	*/*
0x3e			end of payload at 8 + 54 = 62, so the block ends here

Check: the five fields take 6 + 13 + 17 + 12 + 6 = 54 octets, which matches Length.

Frame 2: response HEADERS (server to client, 8 + 84 octets)

Offset	Bytes	Field	Meaning
0x00	00 00 54	Length	0x54 = 84

Offset	Bytes	Field	Meaning
0x03	01	Type	HEADERS
0x04	00	Flags	no END, so DATA frames follow
0x05	00	Reserved	
0x06	00 01	Request ID	1, answering the request above
0x08	03 00 03 32 30 30	:status	200, as three ASCII digits
0x0e	07 00 19 74 65 ... 2d 38	content-type	0x19 = 25 octets: text/plain; charset=utf-8
0x2a	08 00 02 32 31	content-length	21, advisory only
0x2f	09 00 0a 62 73 ... 2e 30	server	bserve/1.0
0x3c	0a 00 1d 54 68 ... 4d 54	date	0x1d = 29 octets: Thu, 08 Oct 2026 16:50:45 GMT
0x5c			end: 8 + 84 = 92

Check: 6 + 28 + 5 + 13 + 32 = 84. Every name came from the static table, so no literal names were needed. With literal names, the same five names would cost 48 more octets.

Frame 3: response DATA (server to client, 8 + 21 octets)

Offset	Bytes	Field	Meaning
0x00	00 00 15	Length	0x15 = 21
0x03	02	Type	DATA
0x04	01	Flags	END: the response, and its body, is complete
0x05	00	Reserved	
0x06	00 01	Request ID	1
0x08	48 65 6c 6c 6f 2c 20 62 69 6e 61 72 79 20 77 6f 72 6c 64 21 0a	Body	Hello, binary world!\n

The client knows the body is finished from the END flag, not from content-length. The connection stays open, and the client's next request would use Request ID 2.

Bonus: a literal name and a skipped frame

The response to DELETE / (status 405) carries a field that is not in the table:

```
00 05 61 6c 6c 6f 77 | 00 09 | 47 45 54 2c 20 48 45 41 44
^  ^ "allow"         | len 9 | "GET, HEAD"
| NameLen = 5
Index 0 = literal name
```

This frame is one that every v1 receiver skips without error. Its type 0x7f is unknown and its payload length is 3:

```
00 00 03 | 7f | 00 | 00 | 00 00 | 61 62 63
```